

Advanced SQL Concepts & Data Manipulation

Day 2 practical workbook · MySQL 8.0 · Sales order dataset

Day 1 turned the sales spreadsheet into a database of four related tables. Day 2 puts that database to work: sorting, filtering and summarising the data, combining the tables, changing rows, and packaging queries so they can be reused. As before, the statements were run on MySQL 8.0 and the results printed here are the real output.

Part	Topic	What you do
	Quick checks	Confirm the four tables are loaded before starting
1	Sorting with ORDER BY	Sort by several columns, by a calculation and in a custom order
2	Filtering rows	Combine conditions with AND, OR, NOT, IN and BETWEEN
3	Limiting results	Take the top rows, or the next page of rows
4	Aggregates and GROUP BY	Count, total and average per group; filter groups with HAVING
5	Mathematical operations	Calculate new columns; apply rules with CASE
6	Order of operations	See the order in which MySQL processes a query
7	Subqueries	Use the result of one query inside another
8	Joins	Combine tables with INNER, LEFT and RIGHT JOIN
9	Updating and deleting	Change and remove rows safely
10	Stored procedures	Save a query on the server and call it by name
11	ETL with SQL	Build a summary table from the detailed tables
12	Simple statistics	Describe a column with mean, range and standard deviation

Before you start: quick checks

Everything today depends on the four tables built on Day 1: `customers`, `products`, `orders` and `order_items`. Run these three checks before anything else. They take a few seconds and tell you whether your database is ready.

Check 1: the tables exist.

```
USE sales_db;
SHOW TABLES;
```

RESULT**Tables_in_sales_db**

customers
order_items
orders
products
sales_raw

The four tables must be in the list. `sales_raw` is the landing table from Day 1; it is not used today.

Check 2: every table has its rows. UNION ALL stacks the four counts into one result.

```
SELECT 'customers' AS table_name, COUNT(*) AS row_count FROM customers
UNION ALL
SELECT 'products', COUNT(*) FROM products
UNION ALL
SELECT 'orders', COUNT(*) FROM orders
UNION ALL
SELECT 'order_items', COUNT(*) FROM order_items;
```

RESULT

table_name	row_count
customers	730
products	1615
orders	1980
order_items	4000

Check 3: the values are right. Row counts can match while values are wrong, so compare the totals too.

```
SELECT SUM(sales) AS total_sales,
       SUM(profit) AS total_profit
FROM order_items;
```

RESULT

total_sales	total_profit
923303.1530	96112.0837

Watch out: If a table is missing, a count differs or the totals do not match, open `sql_day2_setup.sql` in Workbench (File > Open SQL Script) and run all of it with Ctrl+Shift+Enter. It deletes `sales_db`, rebuilds it, inserts the 4,000 rows and fills the four tables. Then run the three checks again.

`sql_day2_class_script.sql` holds every statement in this workbook in order, starting with these checks.

Part 1 Sorting data with ORDER BY

ORDER BY sorts the result. ASC (ascending) is the default and DESC reverses it. Day 1 sorted by one column; real questions usually need more.

Sorting by more than one column

List the columns in order of importance. The second column only decides the order among rows that tie on the first. Here: largest quantity first, and among equal quantities, largest sale first.

```
SELECT order_id, quantity, sales
FROM order_items
ORDER BY quantity DESC, sales DESC
LIMIT 5;
```

RESULT

order_id	quantity	sales
CA-2014-120768	14	1819.8600
CA-2017-123491	14	1702.1200
CA-2014-154599	14	1075.0880
CA-2015-146563	14	724.0800
US-2015-164448	14	427.4200

Sorting by a calculation

ORDER BY accepts a function or expression, not only a column name. ABS gives the size of a number without its sign, so this finds the biggest swings in profit, whether gain or loss:

```
SELECT order_id, product_id, profit
FROM order_items
ORDER BY ABS(profit) DESC
LIMIT 5;
```

RESULT

order_id	product_id	profit
CA-2017-127180	TEC-CO-10004722	3919.9888
US-2017-168116	TEC-MA-10004125	-3839.9904
CA-2017-134845	TEC-MA-10000822	-3399.9800
CA-2015-145352	OFF-BI-10003527	3177.4750
CA-2015-147830	TEC-MA-10000418	-2639.9912

Sorting in a custom order with CASE

Sorted alphabetically, the ship modes come out as First Class, Same Day, Second Class, Standard Class, which is not fastest to slowest. CASE assigns each value a number and the sort uses that number.

```
SELECT DISTINCT ship_mode
FROM orders
ORDER BY CASE ship_mode
           WHEN 'Same Day'      THEN 1
           WHEN 'First Class'   THEN 2
           WHEN 'Second Class'  THEN 3
           ELSE 4
END;
```

RESULT

ship_mode
Same Day
First Class
Second Class
Standard Class

Watch out: Empty values (NULL) sort first when ascending and last when descending. Some database systems let you write `NULLS LAST` to change that. MySQL does not: it is a syntax error.

Part 2 Filtering rows

WHERE keeps the rows that pass a test. These are the building blocks:

Operator	Meaning	Example
= <>	Equal to, not equal to	region = 'West'
> >= < <=	Greater or less than	sales >= 1000
AND	Both conditions must be true	region = 'West' AND ship_mode = 'Same Day'
OR	At least one condition must be true	state = 'Texas' OR state = 'Florida'
NOT	Reverses a condition	NOT region = 'West'
IN (...)	Matches any value in a list	state IN ('Texas', 'Florida')
BETWEEN a AND b	Within a range, both ends included	sales BETWEEN 100 AND 200
LIKE	Matches a text pattern; % stands for any text	product_name LIKE 'Canon%'

AND: every condition must hold

```
SELECT order_id, order_date, city
FROM orders
WHERE region = 'West'
      AND ship_mode = 'Same Day';
```

RESULT

order_id	order_date	city
CA-2014-106719	2014-08-24	Billings
CA-2014-117317	2014-10-19	Los Angeles
CA-2014-123295	2014-07-18	Tucson

First 3 of 31 rows shown.

OR, and the trap when it is mixed with AND

Question: how many Same Day orders went to Texas or Florida? This looks right and is wrong:

```
SELECT COUNT(*) AS orders
FROM orders
WHERE state = 'Texas' OR state = 'Florida' AND ship_mode = 'Same Day';
```

RESULT

orders
201

MySQL applies AND before OR, the same way multiplication comes before addition. So it read the test as: Texas (any ship mode), or Florida and Same Day. Brackets say what you mean:

```
SELECT COUNT(*) AS orders
FROM orders
WHERE (state = 'Texas' OR state = 'Florida')
      AND ship_mode = 'Same Day';
```

RESULT

orders
18

IN: a tidier OR

When several OR tests compare the same column, IN is shorter and avoids the bracket problem altogether. This returns the same 18 orders:

```
SELECT COUNT(*) AS orders
FROM orders
WHERE state IN ('Texas', 'Florida')
      AND ship_mode = 'Same Day';
```

RESULT

orders
18

NOT: everything except

NOT reverses a test. It is most useful with IN:

```
SELECT DISTINCT ship_mode
FROM orders
WHERE ship_mode NOT IN ('Standard Class', 'Second Class');
```

RESULT

ship_mode
First Class
Same Day

BETWEEN: a range

BETWEEN includes both ends, which makes it a natural fit for dates. Orders placed in December 2017:

```
SELECT COUNT(*) AS december_orders
FROM orders
WHERE order_date BETWEEN '2017-12-01' AND '2017-12-31';
```

RESULT

december_orders
101

Part 3 Limiting results with LIMIT

LIMIT cuts the result to a number of rows. On its own it returns whichever rows come first, so pair it with ORDER BY to make "first" mean something. The three largest order lines:

```
SELECT order_id, product_id, sales
FROM order_items
ORDER BY sales DESC
LIMIT 3;
```

RESULT

order_id	product_id	sales
CA-2014-145317	TEC-MA-10002412	22638.4800
CA-2017-127180	TEC-CO-10004722	11199.9680
CA-2014-143917	OFF-SU-10000151	8187.6500

OFFSET skips rows before the limit is applied. This returns the next three, which is how applications show results one page at a time.

```
SELECT order_id, product_id, sales
FROM order_items
ORDER BY sales DESC
LIMIT 3 OFFSET 3;
```

RESULT

order_id	product_id	sales
CA-2014-139892	TEC-MA-10000822	8159.9520
US-2017-168116	TEC-MA-10004125	7999.9800
CA-2015-145352	OFF-BI-10003527	6354.9500

TRY IT YOURSELF

(a) List the order ID, ship date and city of the five most recently shipped orders in the East region. (b) How many orders in the East region were shipped First Class or Same Day?

Part 4 Aggregate functions and GROUP BY

An aggregate function takes many rows and returns one value.

Function	Returns
COUNT(*)	The number of rows
COUNT(DISTINCT column)	The number of different values in the column
SUM(column)	The total
AVG(column)	The average
MIN(column), MAX(column)	The smallest and largest value

Used on a whole table, they summarise it in a single row:

```
SELECT COUNT(*)           AS order_lines,
       COUNT(DISTINCT product_id) AS products_sold,
       SUM(sales)          AS total_sales,
       AVG(sales)          AS average_sale,
       MAX(sales)          AS largest_sale
FROM order_items;
```

RESULT

order_lines	products_sold	total_sales	average_sale	largest_sale
4000	1615	923303.1530	230.82578825	22638.4800

GROUP BY: one row per group

GROUP BY splits the rows into groups, and the aggregates are then worked out for each group separately. Orders per region:

```
SELECT region, COUNT(*) AS orders
FROM orders
GROUP BY region
ORDER BY orders DESC;
```

RESULT

region	orders
West	609
East	559
Central	478
South	334

Several aggregates can sit side by side. For each ship mode, the number of orders and the average number of days between ordering and shipping:

```
SELECT ship_mode,
       COUNT(*) AS orders,
       AVG(DATEDIFF(ship_date, order_date)) AS avg_days_to_ship
FROM orders
GROUP BY ship_mode
ORDER BY avg_days_to_ship;
```

RESULT

ship_mode	orders	avg_days_to_ship
Same Day	98	0.0408
First Class	318	2.1698
Second Class	393	3.2392
Standard Class	1171	5.0256

Grouping by two columns

With two columns in GROUP BY, there is one group for each combination.

```
SELECT region, ship_mode, COUNT(*) AS orders
FROM orders
GROUP BY region, ship_mode
ORDER BY region, orders DESC;
```

RESULT

region	ship_mode	orders
Central	Standard Class	284
Central	Second Class	95
Central	First Class	76
Central	Same Day	23
East	Standard Class	342
East	Second Class	100

First 6 of 16 rows shown.

The rule that catches everyone

Every column in the SELECT list must either appear in GROUP BY or be inside an aggregate function. Here state is neither:

```
SELECT region, state, COUNT(*) AS orders
FROM orders
GROUP BY region;
```

ERROR

```
Error Code: 1055. Expression #2 of SELECT list is not in GROUP BY clause and contains nonaggregated column 'sales_db.orders.state' which is not functionally dependent on columns in GROUP BY clause; this is incompatible with sql_mode=only_full_group_by
```

Each region contains many states, so MySQL cannot know which one to show on the region's single row. Either add state to the GROUP BY or remove it from the SELECT.

HAVING: filtering groups

WHERE cannot test an aggregate, because it runs before the groups exist. HAVING does that job: it filters the groups after they have been formed. Customers with eight or more orders:

```
SELECT customer_id, COUNT(*) AS orders
FROM orders
GROUP BY customer_id
HAVING COUNT(*) >= 8
ORDER BY orders DESC;
```

RESULT

customer_id	orders
JE-15745	10
BM-11650	8
EP-13915	8

The two work together. WHERE first narrows the rows to 2017, then the groups are formed, then HAVING keeps the groups with five or more orders:

```
SELECT customer_id, COUNT(*) AS orders_in_2017
FROM orders
WHERE order_date >= '2017-01-01'
GROUP BY customer_id
HAVING COUNT(*) >= 5;
```

RESULT

customer_id	orders_in_2017
EP-13915	5
LW-16990	5
TD-20995	5

TRY IT YOURSELF

(a) Show the number of orders per state, largest first, top five only. (b) Which states have more than 150 orders?

Part 5 Mathematical operations

Columns can be calculated in the `SELECT` list with `+`, `-`, `*` and `/`. `ROUND(value, n)` rounds to `n` decimals. Here unit price is sales divided by quantity, and margin is profit as a percentage of sales:

```
SELECT order_id,
       sales,
       quantity,
       profit,
       ROUND(sales / quantity, 2) AS unit_price,
       ROUND(profit / sales * 100, 1) AS margin_pct
FROM order_items
ORDER BY sales DESC
LIMIT 4;
```

RESULT

order_id	sales	quantity	profit	unit_price	margin_pct
CA-2014-145317	22638.4800	6	-1811.0784	3773.08	-8.0
CA-2017-127180	11199.9680	4	3919.9888	2799.99	35.0
CA-2014-143917	8187.6500	5	327.5060	1637.53	4.0
CA-2014-139892	8159.9520	8	-1359.9920	1019.99	-16.7

Multiplication and division are done before addition and subtraction. Use brackets whenever the order matters: `(sales - profit) / quantity` is the cost per unit, while `sales - profit / quantity` is something else.

Calculating with aggregates

Arithmetic also works on aggregates. The overall margin is total profit divided by total sales:

```
SELECT ROUND(SUM(profit) / SUM(sales) * 100, 1) AS overall_margin_pct
FROM order_items;
```

RESULT

overall_margin_pct
10.4

CASE: a calculation with rules

CASE returns a different value depending on a condition. It checks each WHEN from the top and stops at the first one that is true. On Day 1 the spreadsheet's SalesCategory column was left out of the design because it can be calculated. This is that calculation:

```
SELECT order_id,
       sales,
       CASE
         WHEN sales > 1000 THEN 'High'
         WHEN sales > 100  THEN 'Medium'
         ELSE 'Low'
       END AS sales_band
FROM order_items
LIMIT 4;
```

RESULT

order_id	sales	sales_band
CA-2016-152156	261.9600	Medium
CA-2016-152156	731.9400	Medium
CA-2016-138688	14.6200	Low
US-2015-108966	957.5775	Medium

A calculated column can be grouped like any other. How many order lines fall in each band?

```
SELECT CASE
  WHEN sales > 1000 THEN 'High'
  WHEN sales > 100  THEN 'Medium'
  ELSE 'Low'
END AS sales_band,
COUNT(*) AS order_lines
FROM order_items
GROUP BY sales_band
ORDER BY order_lines DESC;
```

RESULT

sales_band	order_lines
Low	2484
Medium	1315
High	201

TRY IT YOURSELF

Label each order line Loss when its profit is below zero and Profit otherwise, then count the order lines under each label.

Part 6 Order of operations

A query is written in one order and processed in another. MySQL works through the clauses like this:

Step	Clause	What happens
1	FROM, JOIN	The tables are read and combined
2	WHERE	Rows that fail the test are dropped

Step	Clause	What happens
3	GROUP BY	The remaining rows are put into groups
4	HAVING	Groups that fail the test are dropped
5	SELECT	The output columns are calculated and aliases are assigned
6	ORDER BY	The result is sorted
7	LIMIT	The result is cut to the number of rows asked for

The numbers in the comments below show the order in which each line is processed:

```

SELECT region, COUNT(*) AS same_day_orders    -- 5
FROM orders                                   -- 1
WHERE ship_mode = 'Same Day'                  -- 2
GROUP BY region                               -- 3
HAVING COUNT(*) >= 20                          -- 4
ORDER BY same_day_orders DESC                 -- 6
LIMIT 2;                                       -- 7

```

RESULT

region	same_day_orders
West	31
East	28

This order explains three rules you have already met:

- An alias cannot be used in WHERE (step 2) because aliases are only assigned in SELECT (step 5). It can be used in ORDER BY (step 6).
- An aggregate cannot be used in WHERE because the groups do not exist until step 3. Test aggregates in HAVING.
- LIMIT is applied last, so it always cuts the sorted result.

The second rule in action:

```

SELECT customer_id, COUNT(*) AS orders
FROM orders
WHERE COUNT(*) >= 8
GROUP BY customer_id;

```

ERROR

```
Error Code: 1111. Invalid use of group function
```

Part 7 Subqueries

A subquery is a query written inside another query, in brackets. The inner query runs first and the outer query uses its result.

A subquery that returns one value

Which order lines are larger than the average sale? The average is not known in advance, so let a subquery work it out:

```
SELECT order_id, product_id, sales
FROM order_items
WHERE sales > (SELECT AVG(sales) FROM order_items)
ORDER BY sales;
```

RESULT

order_id	product_id	sales
US-2015-149629	FUR-BO-10004709	231.9200
CA-2016-111416	OFF-BI-10002026	232.4000
CA-2016-142097	OFF-BI-10002082	232.9600

First 3 of 967 rows shown.

The inner query returns one number, 230.83, and the outer query compares every row with it.

A subquery that returns a list

With IN, the subquery supplies the list. Customers who placed an order in December 2017:

```
SELECT customer_id, customer_name
FROM customers
WHERE customer_id IN (SELECT customer_id
                      FROM orders
                      WHERE order_date >= '2017-12-01')
ORDER BY customer_name;
```

RESULT

customer_id	customer_name
AH-10030	Aaron Hawkins
AS-10090	Adam Shillingsburg
AB-10165	Alan Barnes

First 3 of 95 rows shown.

A subquery used as a table

A subquery in FROM acts as a temporary table and must be given an alias. The inner query totals each order; the outer query averages those totals to give the average order value. This is the CTE from Day 1 written inline.

```
SELECT ROUND(AVG(order_total), 2) AS average_order_value
FROM (SELECT order_id, SUM(sales) AS order_total
      FROM order_items
      GROUP BY order_id) AS order_totals;
```

RESULT

average_order_value
466.31

A correlated subquery

A correlated subquery refers to the row the outer query is currently looking at, so it is worked out again for every row. Here each order line is compared with the average sale of its own product: `oi.product_id` comes from the outer query.

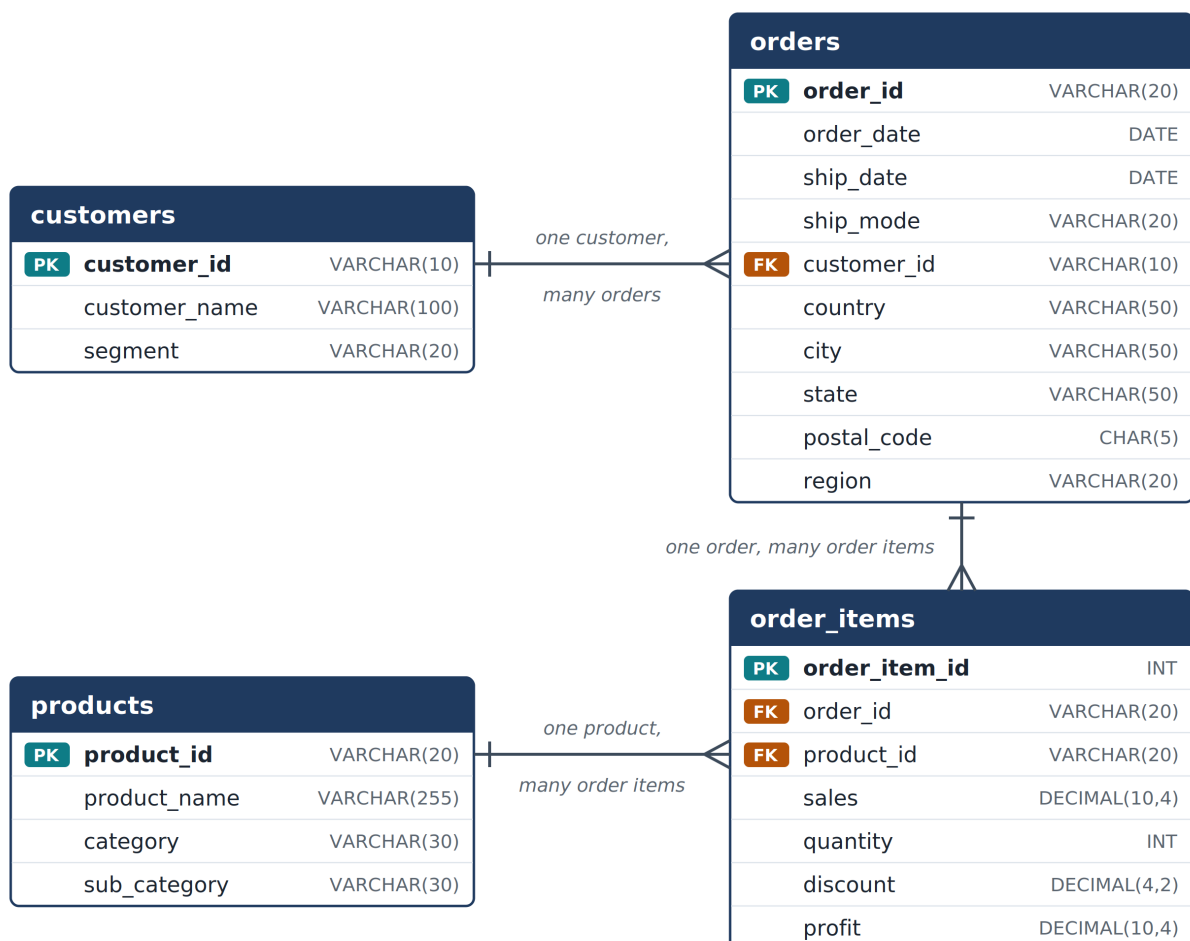
```
SELECT oi.order_id, oi.product_id, oi.sales
FROM order_items AS oi
WHERE oi.sales > (SELECT AVG(sales)
                  FROM order_items
                  WHERE product_id = oi.product_id)
ORDER BY oi.sales DESC
LIMIT 3;
```

RESULT

order_id	product_id	sales
CA-2014-139892	TEC-MA-10000822	8159.9520
CA-2015-145352	OFF-BI-10003527	6354.9500
CA-2017-158379	OFF-SU-10002881	4663.7360

Part 8 Joins

The design from Day 1 stores each fact once, so an answer often needs columns from more than one table. `orders` holds `customer_id` but not the customer's name. A join follows the foreign key to fetch it.



INNER JOIN: rows that match in both tables

```
SELECT o.order_id, o.order_date, c.customer_name
FROM orders AS o
INNER JOIN customers AS c ON c.customer_id = o.customer_id
ORDER BY o.order_date DESC, o.order_id
LIMIT 3;
```

RESULT

order_id	order_date	customer_name
CA-2017-115427	2017-12-30	Erica Bern
CA-2017-126221	2017-12-30	Chuck Clark
CA-2017-143259	2017-12-30	Patrick O'Donnell

- ON says how the rows match: the foreign key in one table equals the primary key in the other.
- The table aliases o and c say which table each column comes from. A column name that exists in both tables, such as customer_id, must be given its alias or MySQL reports Error Code: 1052. Column is ambiguous.
- JOIN on its own means INNER JOIN.

Joining more than two tables

Add one JOIN for each extra table. This puts the first order in the spreadsheet back together from all four tables:

```
SELECT o.order_id, c.customer_name, p.product_name, oi.quantity, oi.sales
FROM orders AS o
JOIN customers AS c ON c.customer_id = o.customer_id
JOIN order_items AS oi ON oi.order_id = o.order_id
JOIN products AS p ON p.product_id = oi.product_id
WHERE o.order_id = 'CA-2016-152156';
```

RESULT

order_id	customer_name	product_name	quantity	sales
CA-2016-152156	Claire Gute	Bush Somerset Collection Bookcase	2	261.9600
CA-2016-152156	Claire Gute	Hon Deluxe Fabric Upholstered Stacking Chairs, Rounded Back	3	731.9400

Joining and grouping

Most business questions need both. Sales and profit are in `order_items`, the category is in `products`, so sales by category needs a join and then a `GROUP BY`:

```
SELECT p.category,
       COUNT(*)           AS order_lines,
       SUM(oi.sales)      AS total_sales,
       ROUND(SUM(oi.profit) / SUM(oi.sales) * 100, 1) AS margin_pct
FROM   order_items AS oi
JOIN   products AS p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY total_sales DESC;
```

RESULT

category	order_lines	total_sales	margin_pct
Technology	752	347778.0740	12.8
Furniture	843	293001.2040	2.1
Office Supplies	2405	282523.8750	16.0

LEFT JOIN: keep every row of the first table

An inner join drops rows that have no match. To see the difference, add a customer who has not ordered anything yet:

```
INSERT INTO customers (customer_id, customer_name, segment)
VALUES ('ZZ-00001', 'Thandi Nkosi', 'Consumer');
```

OUTPUT 1 row(s) affected

A `LEFT JOIN` returns every row of the left table (the one named first). Where the right table has no match, its columns are filled with `NULL`.

```
SELECT c.customer_id, c.customer_name, o.order_id
FROM   customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.customer_id
WHERE  c.customer_id IN ('AB-10600', 'ZZ-00001');
```

RESULT

customer_id	customer_name	order_id
AB-10600	Ann Blume	US-2017-155425
ZZ-00001	Thandi Nkosi	NULL

With an inner join the second row would not appear at all. That `NULL` is useful: filtering on it finds the customers who have never ordered.

```
SELECT c.customer_id, c.customer_name
FROM   customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.customer_id
WHERE  o.order_id IS NULL;
```

RESULT

customer_id	customer_name
ZZ-00001	Thandi Nkosi

RIGHT JOIN

A **RIGHT JOIN** is the mirror image: it keeps every row of the table named second. The query below gives the same answer as the one above with the tables swapped. Most people pick **LEFT JOIN** and stick to it.

```
SELECT c.customer_id, c.customer_name
FROM orders AS o
RIGHT JOIN customers AS c ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

RESULT

customer_id	customer_name
ZZ-00001	Thandi Nkosi

Watch out: MySQL has no **FULL OUTER JOIN** (all rows from both tables). Writing it gives a syntax error. It is rarely needed; when it is, a **LEFT JOIN** and a **RIGHT JOIN** are combined with **UNION**.

TRY IT YOURSELF

(a) List the five customers with the highest total sales, showing the customer's name and the total. (b) Show total sales for each sub-category in the **Technology** category.

Part 9 Updating and deleting data

UPDATE changes values in rows that already exist. **DELETE** removes rows. Both act on every row that matches the **WHERE**, so the **WHERE** is the part to get right. Practise on the customer added in Part 8.

UPDATE

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

Before changing anything, run a **SELECT** with the same **WHERE** to see exactly which rows will be affected:

```
SELECT * FROM customers
WHERE customer_id = 'ZZ-00001';
```

RESULT

customer_id	customer_name	segment
ZZ-00001	Thandi Nkosi	Consumer

One row, the right one. Now make the change. Several columns can be set at once, separated by commas.

```
UPDATE customers
SET customer_name = 'Thandi Nkosi-Smith',
    segment = 'Corporate'
WHERE customer_id = 'ZZ-00001';
```

OUTPUT 1 row(s) affected

```
SELECT * FROM customers
WHERE customer_id = 'ZZ-00001';
```

RESULT

customer_id	customer_name	segment
ZZ-00001	Thandi Nkosi-Smith	Corporate

Watch out: An UPDATE or DELETE without a WHERE changes every row in the table, and there is no undo. Workbench's safe update mode blocks statements whose WHERE does not use a key column (Error Code: 1175). Leave it switched on.

DELETE

```
DELETE FROM customers
WHERE customer_id = 'ZZ-00001';
```

OUTPUT 1 row(s) affected

The table is back to its 730 customers. The foreign keys protect the rest: a customer who still has orders cannot be deleted.

```
DELETE FROM customers
WHERE customer_id = 'CG-12520';
```

ERROR

Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails (`sales\_db`.`orders`, CONSTRAINT `orders\_ibfk\_1` FOREIGN KEY (`customer\_id`) REFERENCES `customers` (`customer\_id`))

Part 10 Stored procedures

A stored procedure is a named block of SQL saved on the server. Write the query once, then run it by name with CALL, passing in the values that change each time.

```
DELIMITER //
CREATE PROCEDURE procedure_name (IN parameter_name data_type)
BEGIN
    -- SQL statements
END //
DELIMITER ;
```

Line	Meaning
DELIMITER //	The procedure body contains semicolons. This tells Workbench that a statement now ends at // so it does not stop at the first semicolon.
CREATE PROCEDURE ... (IN ...)	The procedure's name and its parameters. IN marks a value passed in by the caller.
BEGIN ... END //	The body: the statements that run when the procedure is called.
DELIMITER ;	Switches the statement ending back to the semicolon.

A procedure with one parameter

This procedure lists the orders of whichever customer it is given. `DROP PROCEDURE IF EXISTS` comes first so the block can be run again after an edit.

```
DROP PROCEDURE IF EXISTS get_customer_orders;

DELIMITER //
CREATE PROCEDURE get_customer_orders (IN p_customer_id VARCHAR(10))
BEGIN
    SELECT order_id, order_date, ship_mode, city, state
    FROM orders
    WHERE customer_id = p_customer_id
    ORDER BY order_date;
END //
DELIMITER ;
```

```
CALL get_customer_orders('SO-20335');
```

RESULT

order_id	order_date	ship_mode	city	state
US-2015-108966	2015-10-11	Standard Class	Fort Lauderdale	Florida
CA-2015-161718	2015-12-04	Standard Class	Hempstead	New York

Watch out: Give parameters a prefix such as `p_`. If the parameter were also called `customer_id`, the test `WHERE customer_id = customer_id` would compare the parameter with itself, be true for every row, and return all 1,980 orders.

A procedure that summarises

Any query can go inside a procedure. This one reports sales by category for a chosen year:

```
DROP PROCEDURE IF EXISTS category_sales_for_year;

DELIMITER //
CREATE PROCEDURE category_sales_for_year (IN p_year INT)
BEGIN
    SELECT p.category,
           COUNT(DISTINCT o.order_id) AS orders,
           SUM(oi.sales) AS total_sales
    FROM orders AS o
    JOIN order_items AS oi ON oi.order_id = o.order_id
    JOIN products AS p ON p.product_id = oi.product_id
    WHERE YEAR(o.order_date) = p_year
    GROUP BY p.category
    ORDER BY total_sales DESC;
END //
DELIMITER ;
```

```
CALL category_sales_for_year(2017);
```

RESULT

category	orders	total_sales
Technology	206	107674.1470
Office Supplies	489	99408.3420
Furniture	222	80493.2345

Call it again with 2016 and the same code answers a different question. To list the procedures in the database or remove one:

```
SHOW PROCEDURE STATUS WHERE Db = 'sales_db';
```

```
DROP PROCEDURE IF EXISTS procedure_name;
```

TRY IT YOURSELF

Write a procedure called `orders_in_state` that takes a state name and returns the order ID, order date and city of every order shipped to that state. Call it for Vermont.

Part 11 ETL with SQL

ETL stands for Extract, Transform, Load: get data out of a source, reshape it, and store the result where it is needed. You have already done one. Day 1 was an ETL job:

Stage	Means	What we did on Day 1
Extract	Read data from its source	Loaded the CSV file into <code>sales_raw</code>
Transform	Clean and reshape it	Removed repeats with <code>DISTINCT</code> , chose one name per product, repaired postal codes with <code>LPAD</code>
Load	Store the result	Filled the four tables with <code>INSERT ... SELECT</code>

ETL also runs inside a database. Analysts often want one tidy row per customer, not thousands of order lines, so today's job builds a `customer_summary` table from the detailed tables.

Step 1: write the transformation as a SELECT

Get the query right before storing anything. `COUNT(DISTINCT o.order_id)` is needed because the join to `order_items` repeats each order once per line.

```
SELECT c.customer_id,
       c.customer_name,
       c.segment,
       COUNT(DISTINCT o.order_id) AS orders,
       SUM(oi.sales)              AS total_sales,
       SUM(oi.profit)             AS total_profit,
       MAX(o.order_date)         AS last_order
FROM   customers AS c
JOIN   orders     AS o ON o.customer_id = c.customer_id
JOIN   order_items AS oi ON oi.order_id = o.order_id
GROUP BY c.customer_id, c.customer_name, c.segment
ORDER BY total_sales DESC
LIMIT 3;
```

RESULT

customer_id	customer_name	segment	orders	total_sales	total_profit	last_order
SM-20320	Sean Miller	Home Office	3	24195.6460	-1755.4565	2017-10-12
TA-21385	Tom Ashbrook	Home Office	1	13716.4580	4597.1657	2017-10-22
KL-16645	Ken Lonsdale	Consumer	6	12167.6670	298.0212	2016-06-04

Step 2: create the target table and load it

`DROP TABLE IF EXISTS` makes the job repeatable: run it again next week and the summary is rebuilt from the latest data.

```
DROP TABLE IF EXISTS customer_summary;

CREATE TABLE customer_summary (
  customer_id      VARCHAR(10)    PRIMARY KEY,
  customer_name    VARCHAR(100)   NOT NULL,
  segment          VARCHAR(20)    NOT NULL,
  orders           INT             NOT NULL,
  total_sales      DECIMAL(12,4)  NOT NULL,
  total_profit     DECIMAL(12,4)  NOT NULL,
  last_order       DATE            NOT NULL
);
```

Then load it with the query from Step 1, minus the `ORDER BY` and `LIMIT`:

```
INSERT INTO customer_summary
(customer_id, customer_name, segment, orders,
 total_sales, total_profit, last_order)
SELECT c.customer_id,
       c.customer_name,
       c.segment,
       COUNT(DISTINCT o.order_id),
       SUM(oi.sales),
       SUM(oi.profit),
       MAX(o.order_date)
FROM customers AS c
JOIN orders    AS o ON o.customer_id = c.customer_id
JOIN order_items AS oi ON oi.order_id = o.order_id
GROUP BY c.customer_id, c.customer_name, c.segment;
```

OUTPUT 730 row(s) affected

Step 3: check the load

The same two checks as this morning: the row count, and a total that must agree with the source.

```
SELECT COUNT(*)      AS customers,
       SUM(orders)    AS orders,
       SUM(total_sales) AS total_sales
FROM customer_summary;
```

RESULT

customers	orders	total_sales
730	1980	923303.1530

730 customers, 1,980 orders and the same total sales as Check 3. Questions about customers are now simple queries on one table, with no joins:

```
SELECT segment,
       COUNT(*)           AS customers,
       ROUND(AVG(total_sales), 2) AS avg_sales_per_customer
FROM customer_summary
GROUP BY segment
ORDER BY avg_sales_per_customer DESC;
```

RESULT

segment	customers	avg_sales_per_customer
Home Office	136	1477.28
Corporate	212	1283.22
Consumer	382	1178.93

Part 12 Analysing data with simple statistics

Aggregate functions are also the tools of descriptive statistics. MySQL adds two for spread: `STDDEV_SAMP` (sample standard deviation) and `VAR_SAMP` (sample variance).

```
SELECT COUNT(*)           AS n,
       ROUND(AVG(sales), 2) AS mean,
       MIN(sales)          AS minimum,
       MAX(sales)          AS maximum,
       MAX(sales) - MIN(sales) AS value_range,
       ROUND(STDDEV_SAMP(sales), 2) AS std_dev
FROM order_items;
```

RESULT

n	mean	minimum	maximum	value_range	std_dev
4000	230.83	0.8760	22638.4800	22637.6040	625.5

Read the numbers together. The mean sale is about 231, but the standard deviation is far larger than the mean and the maximum is over 22,000. Most order lines are small and a few very large ones pull the average up, so the mean alone gives a misleading picture of a typical sale.

Statistics per group

Add `GROUP BY` to compare groups. The category comes from `products`, so this needs a join:

```
SELECT p.category,
       COUNT(*)           AS n,
       ROUND(AVG(oi.sales), 2) AS mean,
       MIN(oi.sales)       AS minimum,
       MAX(oi.sales)       AS maximum,
       ROUND(STDDEV_SAMP(oi.sales), 2) AS std_dev
FROM order_items AS oi
JOIN products AS p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY mean DESC;
```

RESULT

category	n	mean	minimum	maximum	std_dev
Technology	752	462.47	0.9900	22638.4800	1157.34
Furniture	843	347.57	1.9880	3610.8480	455.11
Office Supplies	2405	117.47	0.8760	8187.6500	354.17

Technology has the highest average sale and by far the widest spread. Office Supplies has the most order lines and the smallest average.

Tip: `STDDEV_SAMP` and `VAR_SAMP` divide by $n - 1$ and are the right choice when the rows are a sample of something larger. `STDDEV_POP` and `VAR_POP` divide by n , for when the rows are the whole population.

TRY IT YOURSELF

For each region, show the number of order lines, the average sale and the standard deviation of sales. The region is in `orders` and the sales are in `order_items`.

Appendix: Answers to the exercises

Part 3

```
-- (a)
SELECT order_id, ship_date, city
FROM orders
WHERE region = 'East'
ORDER BY ship_date DESC
LIMIT 5;

-- (b)
SELECT COUNT(*) AS orders
FROM orders
WHERE region = 'East'
AND ship_mode IN ('First Class', 'Same Day');
```

Part 4

```
-- (a)
SELECT state, COUNT(*) AS orders
FROM orders
GROUP BY state
ORDER BY orders DESC
LIMIT 5;

-- (b)
SELECT state, COUNT(*) AS orders
FROM orders
GROUP BY state
HAVING COUNT(*) > 150
ORDER BY orders DESC;
```

Part 5

```
SELECT CASE
    WHEN profit < 0 THEN 'Loss'
    ELSE 'Profit'
END AS result,
COUNT(*) AS order_lines
FROM order_items
GROUP BY result;
```

Part 8

```
-- (a)
SELECT c.customer_name, SUM(oi.sales) AS total_sales
FROM customers AS c
JOIN orders AS o ON o.customer_id = c.customer_id
JOIN order_items AS oi ON oi.order_id = o.order_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_sales DESC
LIMIT 5;

-- (b)
SELECT p.sub_category, SUM(oi.sales) AS total_sales
FROM order_items AS oi
JOIN products AS p ON p.product_id = oi.product_id
WHERE p.category = 'Technology'
GROUP BY p.sub_category
ORDER BY total_sales DESC;
```

Part 10

```
DROP PROCEDURE IF EXISTS orders_in_state;

DELIMITER //
CREATE PROCEDURE orders_in_state (IN p_state VARCHAR(50))
BEGIN
    SELECT order_id, order_date, city
    FROM orders
    WHERE state = p_state
    ORDER BY order_date;
END //
DELIMITER ;

CALL orders_in_state('Vermont');
```

Part 12

```
SELECT o.region,
       COUNT(*) AS n,
       ROUND(AVG(oi.sales), 2) AS mean,
       ROUND(STDDEV_SAMP(oi.sales), 2) AS std_dev
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
GROUP BY o.region
ORDER BY mean DESC;
```

Tidying up

To remove what today's session added to the database:

```
DROP TABLE IF EXISTS customer_summary;
DROP PROCEDURE IF EXISTS get_customer_orders;
DROP PROCEDURE IF EXISTS category_sales_for_year;
DROP PROCEDURE IF EXISTS orders_in_state;
```